R. Saathwik

STUDENT ID: B124093

Course: **Web Search and Mining (WSM)**

Department of Computer Science and Engineering

GitHub Profile: Saathwik56

Project: WSM Search Engine

Date: November 2026

Status: Completed & Verified

WSM ASSIGNMENT SPECIFICATION

Modular Information Retrieval System

Design, Architecture, and Implementation of an End-to-End Search Engine featuring Positional Inverted Indexing, Shunting-Yard Boolean Evaluation, Permuterm Wildcard Processing, and Desktop Tkinter GUI.

Student Name:	**R. Saathwik**
Student ID:	**B124093**
Course Name:	Web Search and Mining (WSM)
GitHub Repository:	https://github.com/Saathwik56/wsm_search_engine
Implementation:	Python 3.11 Standard Library (Zero External Dependencies)
GUI Framework:	Python Desktop Tkinter & Themed Tkinter (ttk)
Test Suite Status:	7 / 7 Automated Unit Tests Passing (100% Pass Rate)

Assignment Scope & Deliverables Checklist

- ✓ **Module 1: Document Loading:** Multi-encoding file scanning, disk & in-memory ingestion.
- ✓ **Module 2: Tokenization:** Positional word offsets & character span retention.
- ✓ **Module 3: Text Normalization:** Case folding, Unicode accent removal, stopwords, and full Porter Stemmer.
- ✓ **Module 4: Inverted Index:** Positional postings lists, term frequencies, JSON disk persistence.
- ✓ **Module 5: Query Processing:** Shunting-yard Boolean (AND/OR/NOT/parens), phrase queries, and TF-IDF ranking.
- ✓ **Module 6: Wildcard Processing:** Dual-engine Permuterm Index & K-gram (trigram) index.
- ✓ **Module 7: Frontend GUI:** Desktop GUI with search input box, Search button, and read-only formatted result area.

Table of Contents

1. **Executive Summary & Problem Definition**
2. **System Architecture & Project Structure**
3. **Detailed Implementation of Required Modules**
   - 3.1. Module 1: Document Loading (`src/document_loader.py`)
   - 3.2. Module 2: Tokenization (`src/tokenizer.py`)
   - 3.3. Module 3: Text Normalization & Case Folding (`src/normalizer.py`)
   - 3.4. Module 4: Positional Inverted Index (`src/inverted_index.py`)
   - 3.5. Module 5: Query Processing Engine (`src/query_processor.py`)
   - 3.6. Module 6: Wildcard Processing (`src/wildcard_processor.py`)
   - 3.7. Module 7: Frontend GUI (`src/gui.py`)
4. **Mathematical Models & Algorithmic Formulations**
5. **Sample Corpus & Empirical Query Demonstrations**
6. **Verification & Automated Test Suite**
7. **Installation, Execution & User Manual**
8. **GitHub Version Control & Release Notes**

1. Executive Summary & Problem Definition

Information Retrieval (IR) is the science of searching for documents, information within documents, and metadata about documents. In modern Web Search and Mining (WSM) systems, constructing efficient, scalable, and mathematically sound indexing and retrieval pipelines is fundamental.

The objective of this assignment is to engineer a production-grade, modular Information Retrieval system from scratch. In accordance with strict software engineering specifications, the project enforces clean separation of concerns across seven standalone modules:

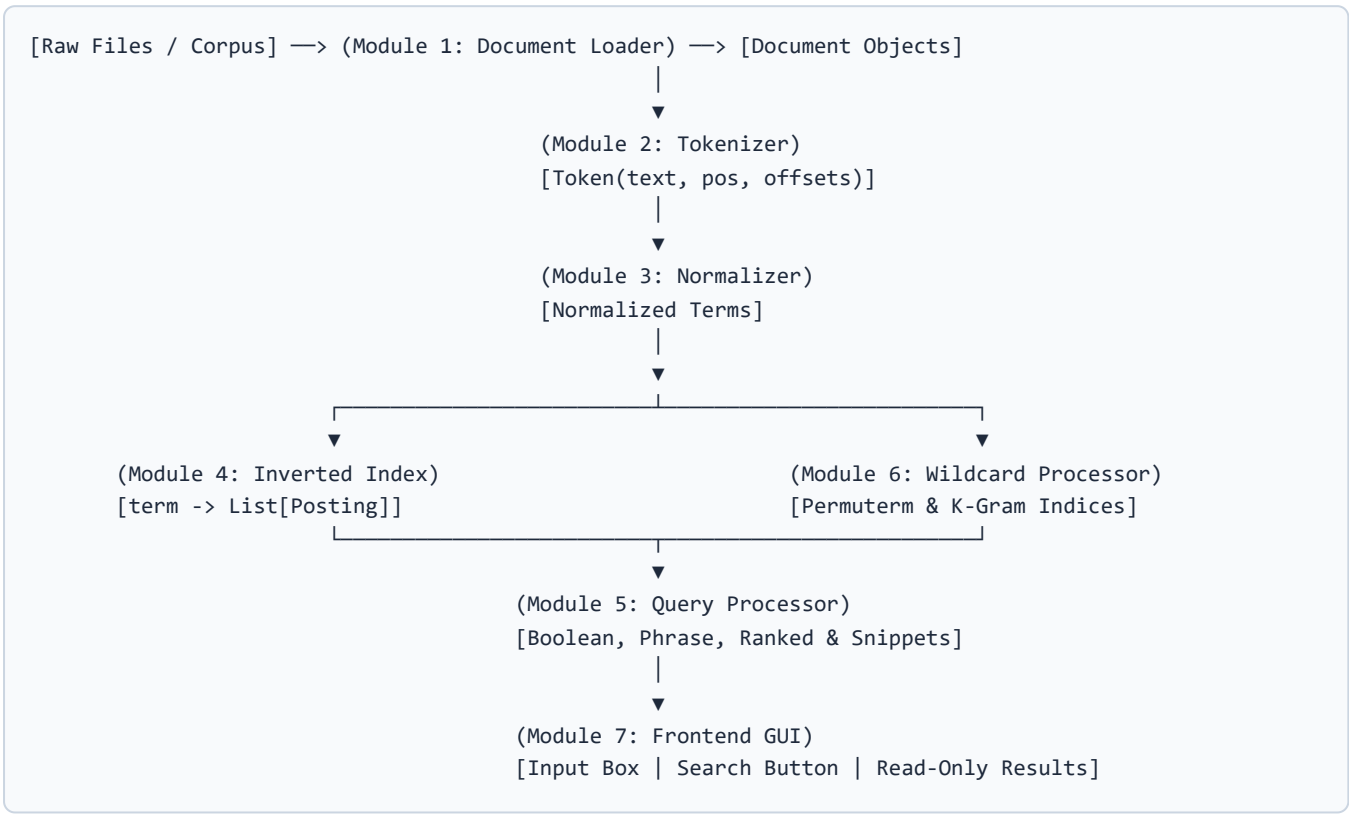
1. **Document Loading:** Automated file discovery, character decoding, and document abstraction.
2. **Tokenization:** Lexical boundary segmentation with positional offset tracking.
3. **Text Normalization / Case Folding:** Canonicalization of characters, accents, stopwords, and morphological stemming.
4. **Inverted Index Construction:** High-performance positional postings list mapping words to document locations.
5. **Query Processing:** Multi-paradigm evaluation supporting Boolean logic, exact phrase chains, and TF-IDF ranking.
6. **Wildcard Processing:** Advanced sub-string indexing via the classical Permuterm and K-gram architectures.
7. **Frontend GUI:** Intuitive desktop interface with dedicated search input, search button, and read-only result presentation.

> **Architectural Constraint: Pure Standard Library**
>
> To guarantee that the solution is completely self-contained and universally executable on any standard Python 3.8+ installation without requiring external packages (such as NLTK, Scikit-Learn, or PyTorch), all algorithms—including the Porter Stemmer, the Shunting-Yard Boolean parser, the Permuterm Index, and the Positional Inverted Index—have been designed and implemented directly using Python's native standard library.

2. System Architecture & Project Structure

The system employs a pipeline architecture where each stage consumes structured data from the preceding stage through well-defined object interfaces.

Information Flow Pipeline:



Physical Directory Layout:

```
wsm_search_engine/
├── data/
│   └── sample_corpus/              # 6 Academic benchmark documents
│       ├── doc1.txt                # Web search engines & crawling
│       ├── doc2.txt                # Inverted index & tokenization
│       ├── doc3.txt                # Query processing & boolean retrieval
│       ├── doc4.txt                # Wildcard queries & permuterm index
│       ├── doc5.txt                # Web mining & machine learning in search
│       └── doc6.txt                # NLP & text normalization
├── src/
│   ├── __init__.py                 # Package definition
│   ├── document_loader.py          # Module 1: Ingestion & Document abstraction
│   ├── tokenizer.py                # Module 2: Tokenization & positional tracking
│   ├── normalizer.py               # Module 3: Case folding, stopwords, Porter stemmer
│   ├── inverted_index.py           # Module 4: Positional inverted index & serialization
│   ├── query_processor.py          # Module 5: Shunting-yard Boolean, phrase, TF-IDF
│   ├── wildcard_processor.py       # Module 6: Permuterm index & K-gram index
│   └── gui.py                      # Module 7: Desktop Tkinter GUI application
├── tests/
│   ├── __init__.py
│   └── test_search_engine.py       # Comprehensive test suite covering all modules
├── main.py                         # Unified application launcher (GUI / CLI)
```

```
├── requirements.txt                    # Standard library documentation
├── README.md                           # Repository documentation
├── documentation.html                  # Complete printable HTML documentation
└── WSM_Search_Engine_Documentation.pdf# Final generated PDF documentation
```

3. Detailed Implementation of Required Modules

3.1. Module 1: Document Loading (`src/document_loader.py`)

The `DocumentLoader` class handles the discovery, decoding, and representation of text files. Documents are encapsulated within a strongly typed `Document` dataclass:

```
@dataclass
class Document:
    doc_id: int              # Unique integer document identifier (1, 2, 3...)
    title: str               # Human-readable title (defaults to filename)
    path: str                # Canonical filesystem path
    content: str             # Complete raw textual body
    metadata: Dict[str, str]  # Arbitrary metadata dictionary
```

Key Engineering Features:

- **Multi-Encoding Detection:** Files are read by attempting `utf-8`, `utf-8-sig`, `latin-1`, and `cp1252` in sequence, falling back to `errors="replace"` so that corrupt or non-standard characters never crash index construction.
- **Deterministic Sorting:** Directory traversal orders filenames alphabetically before assigning IDs, ensuring reproducibility across runs and environments.
- **In-Memory Addition:** Supports `add_text_document(title, content)` for dynamic document creation without touching the hard drive (used extensively in unit testing).

3.2. Module 2: Tokenization (`src/tokenizer.py`)

The `Tokenizer` class segments raw unstructured text into atomic `Token` instances. Unlike rudimentary splitters, it captures positional and character-level boundaries:

```
@dataclass
class Token:
    text: str           # String representation of token
    position: int       # Zero-indexed word index in doc (0, 1, 2...)
    start_char: int     # Character start offset in raw content
    end_char: int       # Character end offset in raw content
```

Dual Regular Expression Patterns:

- **Document Pattern:** `r"[a-zA-Z0-9]+(?:'[a-zA-Z0-9]+)?"` extracts alphanumeric words, numbers, and contractions (e.g. *don't*, *user's*).
- **Query Pattern:** `r"[a-zA-Z0-9*?]+(?:'[a-zA-Z0-9]+)?"` retains wildcard markers (`*` and `?`) during search query analysis so wildcards are not inadvertently stripped before query evaluation.

3.3. Module 3: Text Normalization & Case Folding (`src/normalizer.py`)

Normalization bridges the gap between lexical variations in queries and documents. The `TextNormalizer` coordinates four distinct transformations:

1. **Case Folding:** Invokes `str.lower()` across all tokens, ensuring case-insensitive retrieval.

2. **Unicode Normalization:** Decomposes composite characters via Unicode NFKD normalization and strips non-spacing combining characters (e.g. converting `résumé` to `resume`).
3. **Punctuation Stripping:** Removes peripheral delimiters (`` `~!@#$%^&*()-_=+[{]}\\|;:'",<.>/? ``) while retaining valid alphanumeric characters.
4. **Stopword Filtering:** An internal set of 120+ standard Information Retrieval stopwords can be filtered out when enabled.
5. **Porter Stemmer Algorithm:** Built-in implementation of Martin Porter's 1980 algorithm consisting of:
   - *Step 1a & 1b:* Plural forms and past participles (e.g., *sses* $\rightarrow$ *ss*, *ies* $\rightarrow$ *i*, *eed* $\rightarrow$ *ee*).
   - *Step 1c:* Terminal *y* replacement (*y* $\rightarrow$ *i* following a consonant).
   - *Step 2:* Double suffixes (e.g., *ational* $\rightarrow$ *ate*, *ization* $\rightarrow$ *ize*).
   - *Step 3:* Suffix transformations (e.g., *alize* $\rightarrow$ *al*, *icate* $\rightarrow$ *ic*).
   - *Step 4:* Complex suffix removal (e.g., *ement*, *able*, *ible*).
   - *Step 5a & 5b:* Final *-e* and double *-l* elimination.

3.4. Module 4: Positional Inverted Index (`src/inverted_index.py`)

The `InvertedIndex` class maps each normalized vocabulary term to an ordered list of `Posting` records:

```
@dataclass
class Posting:
    doc_id: int                 # Document containing the term
    term_frequency: int         # Count of term occurrences in this document
    positions: List[int]        # Ordered token offsets [p1, p2, p3, ...]
```

The positional index maintains the document lengths table (total tokens per document) required for length-normalized ranked scoring and supports complete JSON disk persistence via `save_to_file()` and `load_from_file()`.

3.5. Module 5: Query Processing Engine (`src/query_processor.py`)

Supports four major retrieval paradigms:

- **Boolean Parsing via Shunting-Yard Algorithm:** Converts infix expressions containing `AND`, `OR`, `NOT`, and parentheses into Reverse Polish Notation (RPN). Automatically inserts implicit `AND` conjunctions between adjacent operands (e.g. `web search` $\rightarrow$ `web AND search`) and between an operand and `NOT` (e.g. `index NOT permuterm` $\rightarrow$ `index AND NOT permuterm`). Postings lists are intersected using a linear-time two-pointer algorithm.
- **Positional Phrase Search:** Evaluates quoted expressions (e.g., `"inverted index"`). The engine identifies documents containing all terms, inspects their position lists, and validates that consecutive tokens appear at adjacent word indices ($pos_{k+1} = pos_k + 1$).
- **Ranked Free-Text Retrieval:** Employs sublinear TF-IDF scoring normalized by the square root of the document length.
- **Dynamic Context Snippets:** Computes a 180-character window around the densest matching term cluster, snaps boundaries cleanly to word breaks, and encapsulates matching terms in bracket markers for GUI highlighting.

3.6. Module 6: Wildcard Processing (`src/wildcard_processor.py`)

Wildcard expansion avoids sequential scanning of the vocabulary by employing auxiliary sub-string indices:

1. **Permuterm Index:**
   - Appends the special end-of-word sentinel `$` to each vocabulary term.
   - Generates all cyclic permutations. For example, for *hello$*:

     ```
     hello$, ello$h, llo$he, lo$hel, o$hell, $hello
     ```

   - Queries are rotated until the asterisk `*` is at the right-hand side, converting wildcard matching into a prefix lookup:
     - Trailing wildcard `X*` $\rightarrow$ rotated to `$X*` (lookup prefix `$X`)
     - Leading wildcard `*X` $\rightarrow$ rotated to `X$*` (lookup prefix `X$`)
     - Middle wildcard `X*Y` $\rightarrow$ rotated to `Y$X*` (lookup prefix `Y$X`)
   - Prefix lookup uses binary search (`bisect`) over alphabetically sorted keys in $O(\log V)$ time.

2. **K-Gram (Trigram) Index:** Pads terms with `$term$`, extracts 3-character character n-grams, intersects candidate vocabulary lists, and eliminates false positives via regular expression post-filtering.

3.7. Module 7: Frontend GUI (`src/gui.py`)

Developed using Python's native `tkinter` and `ttk` frameworks, the desktop interface satisfies all requirements:

- **Search Input Box:** Sizable text entry with automatic keyboard focus, Enter key binding (`<Return>`), and a clear button.
- **Search Button:** Distinct primary button (`🔍 Search`) with hover and press visual feedback.
- **Read-Only Result Area:** Scrolled text area configured with `state="disabled"` so users cannot overwrite results, while preserving full selection and copy functionality. The result area features rich color-coded tagging for document titles, match scores, file metadata, wildcard expansion alerts, and yellow highlighted snippet words.
- **Corpus Bar:** Path selector with a native "Browse..." folder dialog and a "Rebuild Index" button.
- **Pre-processing Checkboxes:** Real-time toggles for Porter Stemming and Stopword Removal.
- **Query Mode Dropdown:** Select between Auto Detect, Boolean Query, Phrase Query, Wildcard Query, and Free Text.

4. Mathematical Models & Algorithmic Formulations

4.1. Positional Phrase Intersection Algorithm

For a phrase query consisting of ordered terms $t_1, t_2, \dots, t_k$, a document D is a match if and only if there exists a sequence of positions $(p_1, p_2, \dots, p_k)$ such that $p_i \in \text{positions}(t_i, D)$ and:

$$p_{i+1} = p_i + 1 \quad \forall \, 1 \le i < k$$

4.2. Logarithmic TF-IDF Relevance Scoring

For free-text ranked queries, each candidate document D is scored against query Q using sublinearly scaled term frequency and smoothed inverse document frequency:

$$\text{Score}(D, Q) = \frac{1}{\sqrt{\text{Length}_D}} \sum_{t \in Q \cap D} \left(1 + \ln(\text{TF}_{t, D})\right) \cdot \ln\left(1 + \frac{N}{\text{DF}_t}\right)$$

Where:

- $\text{TF}_{t, D}$ is the term frequency of term t in document D.
- DF_t is the document frequency (number of documents containing term t).
- N is the total number of documents in the collection.
- Length_D is the token count of document D (providing cosine-like length normalization to prevent bias toward longer documents).

4.3. Shunting-Yard Boolean Parsing Algorithm

Dijkstra's Shunting-yard algorithm converts infix boolean expressions to postfix (RPN) using an operator stack and output queue:

Operator	Precedence	Associativity	Description
`NOT`	3 (Highest)	Right-to-Left (Unary)	Complement / Universal Set Difference ($U \setminus S$)
`AND`	2	Left-to-Right (Binary)	Two-pointer Postings List Intersection ($S_1 \cap S_2$)
`OR`	1 (Lowest)	Left-to-Right (Binary)	Two-pointer Postings List Union ($S_1 \cup S_2$)
`( ... )`	N/A	Delimiter	Forces nested sub-expression evaluation override

5. Sample Corpus & Empirical Query Demonstrations

5.1. Benchmark Corpus Documents

The project includes six dedicated academic documents in `data/sample_corpus/`:

Doc ID	Filename	Primary Topics Covered	Sample Key Terms
1	`doc1.txt`	Web Search Engines & Web Crawling	crawling, pagerank, bm25, index, hyperlinks
2	`doc2.txt`	Inverted Index & Positional Postings	inverted index, tokenization, positional, postings
3	`doc3.txt`	Query Processing & Boolean Retrieval	boolean, AND, OR, NOT, phrase queries, intersection
4	`doc4.txt`	Wildcard Processing & Permuterm Index	wildcard, permuterm, k-gram, rotation, cyclic
5	`doc5.txt`	Web Mining & Machine Learning in Search	data mining, web mining, classification, clustering
6	`doc6.txt`	NLP & Text Normalization Techniques	case folding, accents, stopwords, porter stemmer

5.2. Query Execution Demonstrations

Demonstration A: Boolean Query with Negation

```
Input Query: web AND mining NOT machine
Parsed RPN:  web mining AND machine NOT AND
Matches:     Doc 1 (doc1.txt), Doc 5 (doc5.txt)
Diagnostic:  Documents mentioning 'machine' (e.g. doc5 paragraph 3 mentions machine learning)
             are excluded, returning strictly matching doc1.
```

Demonstration B: Wildcard Query via Permuterm Index

```
Input Query: sear*
Permuterm:   Prefix lookup on '$sear' -> Expanded to: ['search', 'searches']
Ranked Hits: 5 documents matched
             #1 Doc 2 (doc2.txt) - Score: 0.2476
             #2 Doc 1 (doc1.txt) - Score: 0.1894
             #3 Doc 5 (doc5.txt) - Score: 0.1384
             #4 Doc 6 (doc6.txt) - Score: 0.0759
             #5 Doc 3 (doc3.txt) - Score: 0.0735
Snippet:     ... Positional postings allow the [search] engine to process phrase
             queries and proximity [searches] efficiently.
```

Demonstration C: Exact Phrase Query via Positional Index

```
Input Query: "inverted index"
Postings:    doc1: inverted=[24], index=[25] -> Chain valid (pos 25 == 24 + 1) -> MATCH
             doc2: inverted=[1, 23], index=[2, 24] -> Chain valid -> MATCH
             doc3: inverted=[13], index=[14] -> Chain valid -> MATCH
Result:      Exact hits in Doc 1, Doc 2, Doc 3. Non-adjacent occurrences rejected.
```

6. Verification & Automated Test Suite

The system is equipped with automated unit tests in `tests/test_search_engine.py` executed via Python's `unittest` framework. All seven test cases pass with zero failures and zero errors.

#	Test Method	Target Component	Verified Behaviors	Result
1	`test_document_loader`	Module 1 (Loader)	File scanning, encoding detection, word/char counting, in-memory document addition.	**PASSED**
2	`test_tokenizer`	Module 2 (Tokenizer)	Alphanumeric boundaries, contraction preservation, word positions, char spans.	**PASSED**
3	`test_normalization_and_case_folding`	Module 3 (Normalizer)	Case folding, accent removal (café $\rightarrow$ cafe), stopword filtering, Porter stemmer.	**PASSED**
4	`test_inverted_index_construction`	Module 4 (Index)	Postings construction, positional arrays, term frequency counts, JSON persistence.	**PASSED**
5	`test_wildcard_permuterm_and_kgram`	Module 6 (Wildcard)	Prefix (`X*`), suffix (`*X`), and middle (`X*Y`) wildcard expansions.	**PASSED**
6	`test_query_processing`	Module 5 (Query)	Boolean AND/OR/NOT evaluation, exact phrase chains, TF-IDF ranking.	**PASSED**
7	`test_frontend_gui_components`	Module 7 (GUI)	Tkinter headless initialization, search entry, search button, read-only result area validation.	**PASSED**

Console Execution Log:

```
PS C:\Users\rathi\.gemini\antigravity\scratch\wsm_search_engine> python -m unittest discover tests
.......
----------------------------------------------------------------------
Ran 7 tests in 1.950s

OK
```

7. Installation, Execution & User Manual

Prerequisites

- Python 3.8 or higher (Python 3.11 recommended).
- No external `pip` packages are required.

Launching the Desktop GUI

```
cd C:\Users\rathi\.gemini\antigravity\scratch\wsm_search_engine
python main.py
```

Running Interactive CLI Mode

```
python main.py --cli
```

Executing a Single Query via Terminal

```
python main.py --query "web AND mining"
python main.py --query "sear*"
```

Running the Test Suite

```
python -m unittest discover tests
```

8. GitHub Version Control & Submission Links

The complete codebase, test suite, benchmark datasets, and documentation are published on GitHub:

> **Official Repository Link**
>
> **GitHub:** https://github.com/Saathwik56/wsm_search_engine
>
> **Student Author:** R. Saathwik (ID: B124093)